

Database Laboratory 4

Kassymkozha Yergali

Course: Databases

SQL queries and results for the airport_lab1 database

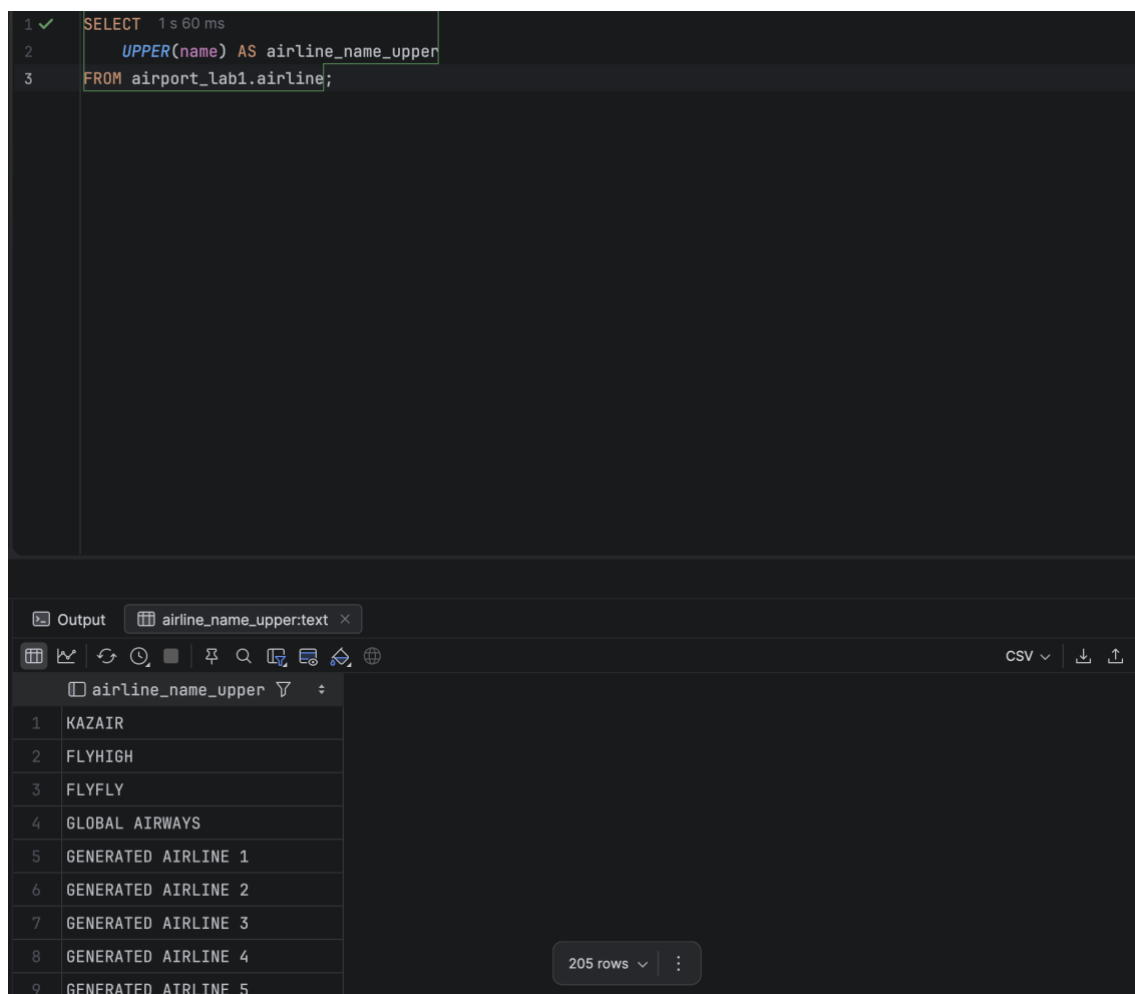
The SQL below is transcribed from the supplied screenshots. All 14 screenshots are preserved.

Task 1 Airline names in uppercase

SQL Query

```
SELECT  
    UPPER(name) AS airline_name_upper  
FROM airport_lab1.airline;
```

Result



The screenshot shows a SQL query execution interface. The query is displayed in a text editor with line numbers 1, 2, and 3. The results are shown in a table below the query. The table has one column, 'airline_name_upper', and 205 rows. The first five rows are: KAZAIR, FLYHIGH, FLYFLY, GLOBAL AIRWAYS, and GENERATED AIRLINE 1. The last row is GENERATED AIRLINE 5. The interface includes a toolbar with various icons and a status bar at the bottom indicating 205 rows.

	airline_name_upper
1	KAZAIR
2	FLYHIGH
3	FLYFLY
4	GLOBAL AIRWAYS
5	GENERATED AIRLINE 1
6	GENERATED AIRLINE 2
7	GENERATED AIRLINE 3
8	GENERATED AIRLINE 4
9	GENERATED AIRLINE 5

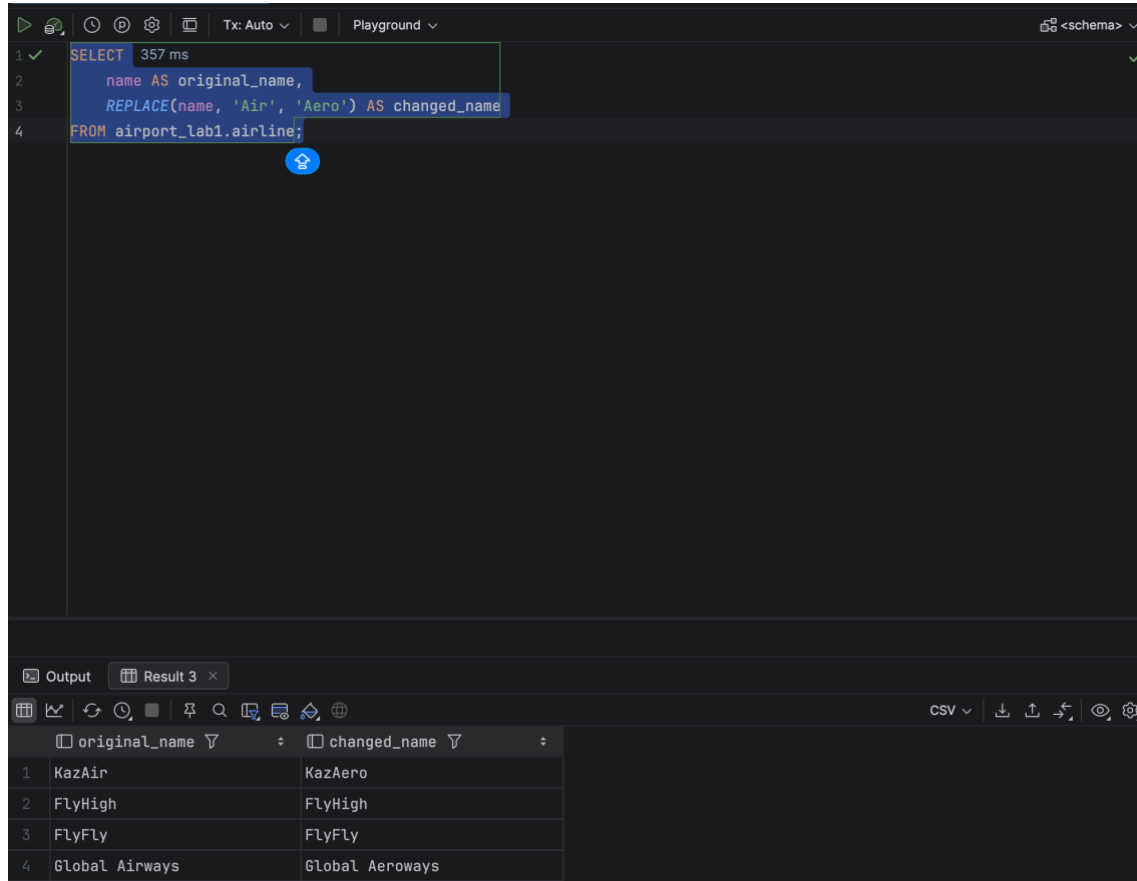
Figure 1. SQL query and result for Task 1.

Task 2 Replacing Air with Aero

SQL Query

```
SELECT
    name AS original_name,
    REPLACE(name, 'Air', 'Aero') AS changed_name
FROM airport_lab1.airline;
```

Result



The screenshot shows a SQL playground interface. The top section displays the SQL query: `SELECT name AS original_name, REPLACE(name, 'Air', 'Aero') AS changed_name FROM airport_lab1.airline;`. The query execution time is 357 ms. Below the query, the results are shown in a table with two columns: `original_name` and `changed_name`. The results are as follows:

	original_name	changed_name
1	KazAir	KazAero
2	FlyHigh	FlyHigh
3	FlyFly	FlyFly
4	Global Airways	Global Aeroways

Figure 2. SQL query and result for Task 2.

Task 3 Flights for airlines 1 and 2

SQL Query

```
SELECT
    flight_id,
    airline_id
FROM airport_lab1.flight
WHERE airline_id IN (1, 2);
```

Result

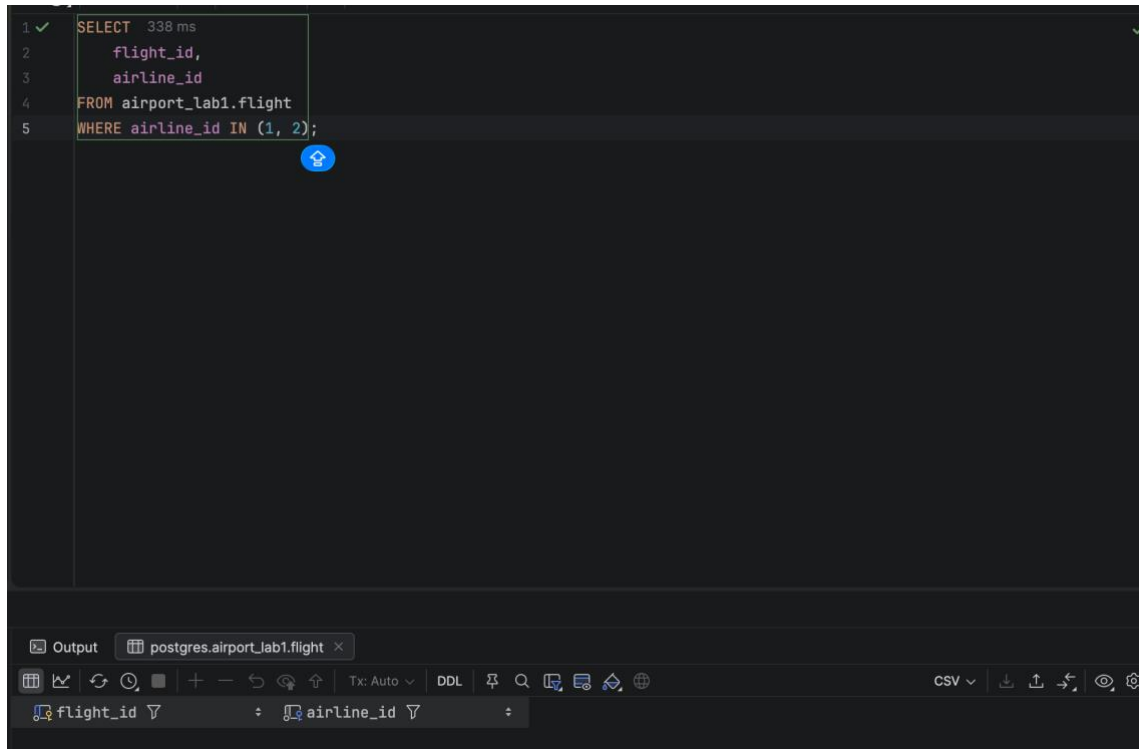


Figure 3. SQL query and result for Task 3.

Task 4 Airport name filters

SQL Query

```
SELECT *  
FROM airport_lab1.airport  
WHERE airport_name ILIKE '%Regional%'  
      AND airport_name ILIKE '%Air%';
```

Result

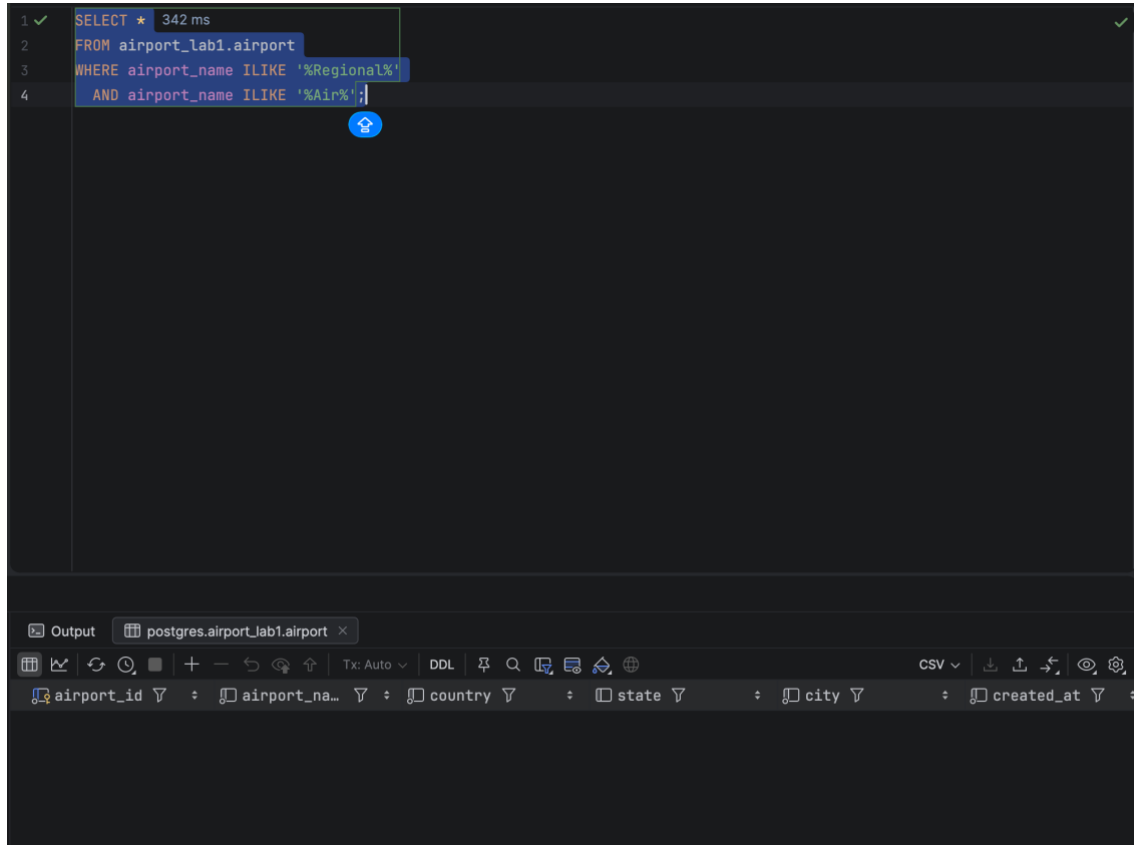


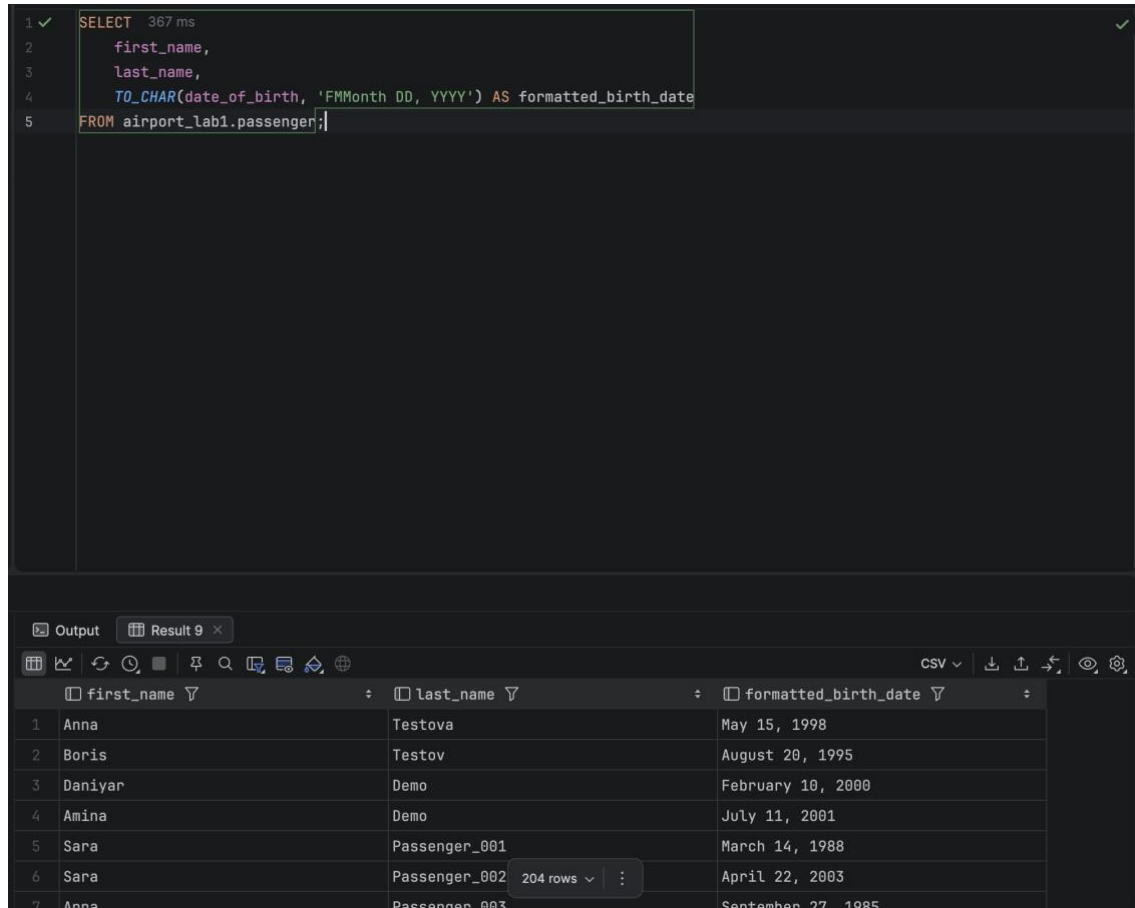
Figure 4. SQL query and result for Task 4.

Task 5 Passenger names and birth dates

SQL Query

```
SELECT
    first_name,
    last_name,
    TO_CHAR(date_of_birth, 'FMMonth DD, YYYY')
    AS formatted_birth_date
FROM airport_lab1.passenger;
```

Result



The screenshot shows a SQL IDE interface. The top pane displays the following SQL query:

```
1 ✓ SELECT 367 ms
2     first_name,
3     last_name,
4     TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS formatted_birth_date
5 FROM airport_lab1.passenger;
```

The bottom pane shows the results of the query in a table format. The table has three columns: first_name, last_name, and formatted_birth_date. The results are as follows:

	first_name	last_name	formatted_birth_date
1	Anna	Testova	May 15, 1998
2	Boris	Testov	August 20, 1995
3	Daniyar	Demo	February 10, 2000
4	Amina	Demo	July 11, 2001
5	Sara	Passenger_001	March 14, 1988
6	Sara	Passenger_002	April 22, 2003
7	Anna	Passenger_003	September 27, 1985

The interface also shows a 'Result 9' tab and a '204 rows' indicator.

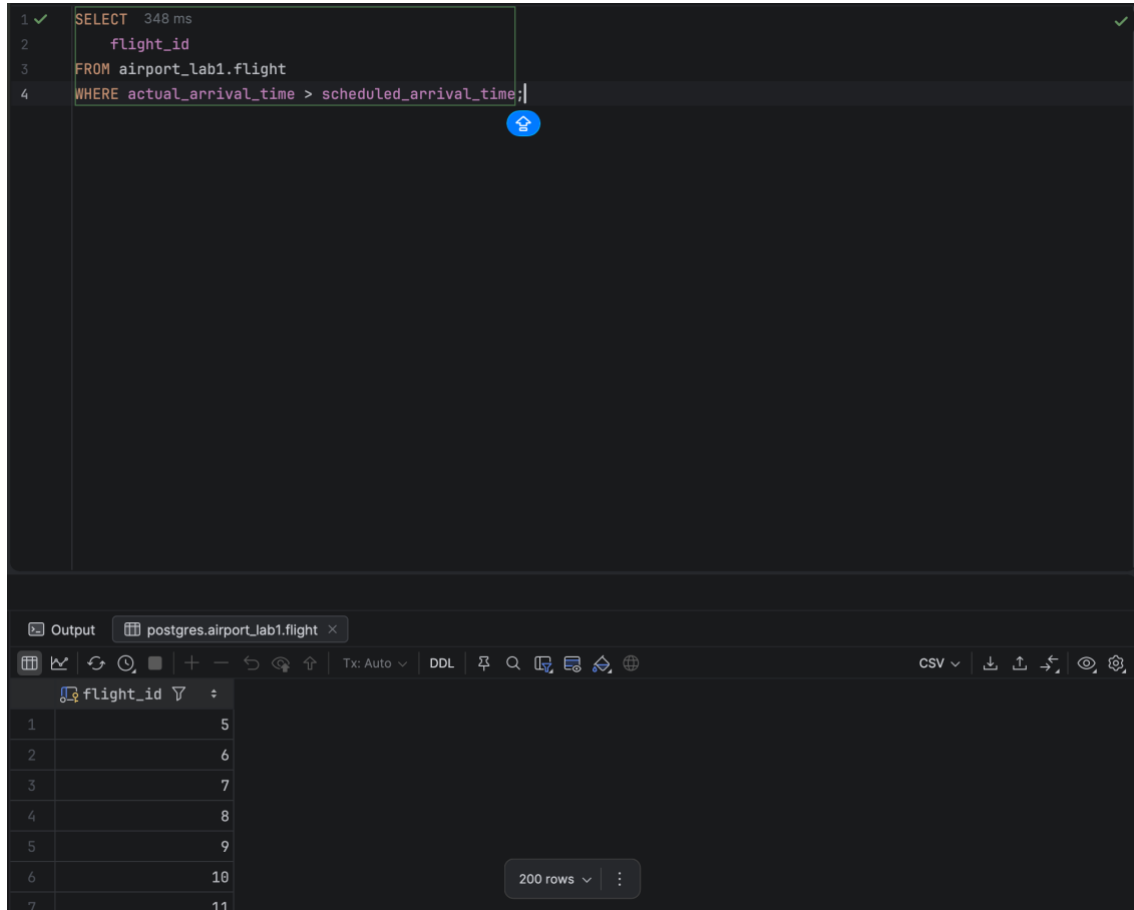
Figure 5. SQL query and result for Task 5.

Task 6 Delayed flight numbers

SQL Query

```
SELECT
    flight_id
FROM airport_lab1.flight
WHERE actual_arrival_time > scheduled_arrival_time;
```

Result



The screenshot shows a SQL query editor with the following query:

```
SELECT
    flight_id
FROM airport_lab1.flight
WHERE actual_arrival_time > scheduled_arrival_time;
```

The query was executed successfully, returning 7 rows of results. The results are displayed in a table with the following data:

flight_id
5
6
7
8
9
10
11

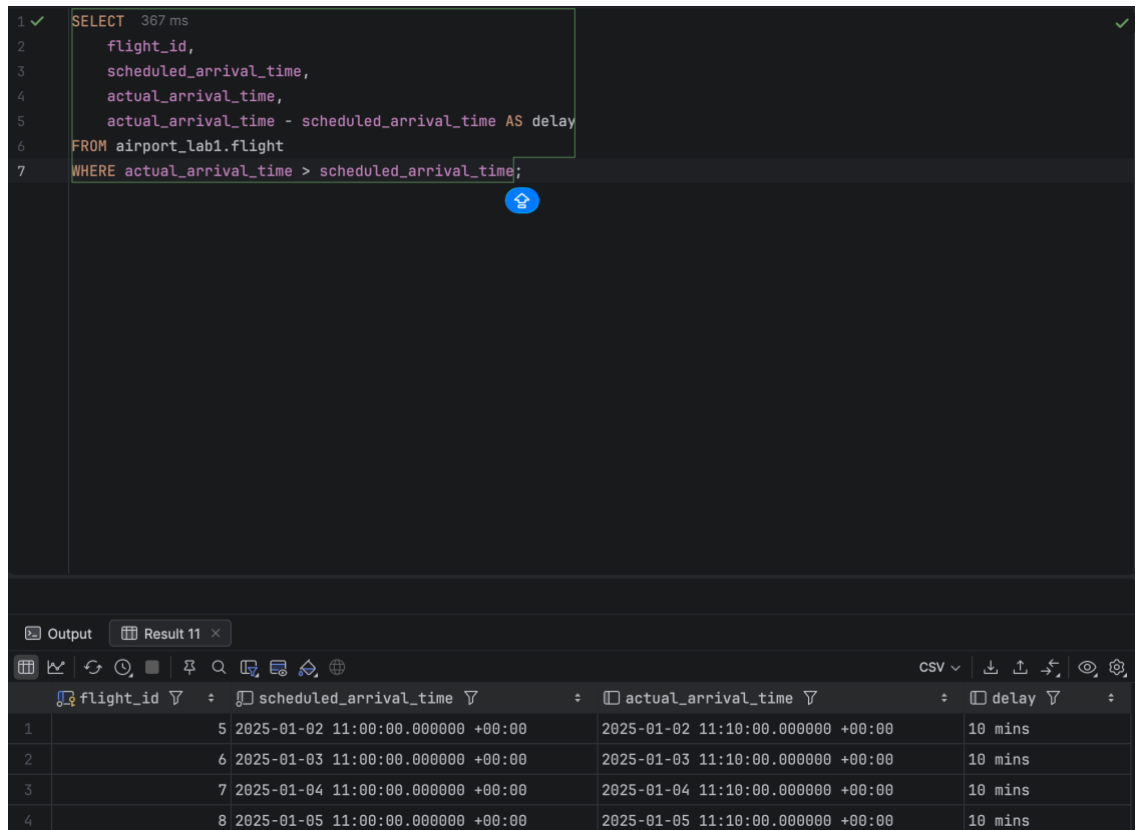
Figure 6. SQL query and result for Task 6.

Task 7 Late arrivals and delay duration

SQL Query

```
SELECT
    flight_id,
    scheduled_arrival_time,
    actual_arrival_time,
    actual_arrival_time - scheduled_arrival_time AS delay
FROM airport_lab1.flight
WHERE actual_arrival_time > scheduled_arrival_time;
```

Result



1 ✓ SELECT 367 ms

2 flight_id,

3 scheduled_arrival_time,

4 actual_arrival_time,

5 actual_arrival_time - scheduled_arrival_time AS delay

6 FROM airport_lab1.flight

7 WHERE actual_arrival_time > scheduled_arrival_time;

Output Result 11

	flight_id	scheduled_arrival_time	actual_arrival_time	delay
1	5	2025-01-02 11:00:00.000000 +00:00	2025-01-02 11:10:00.000000 +00:00	10 mins
2	6	2025-01-03 11:00:00.000000 +00:00	2025-01-03 11:10:00.000000 +00:00	10 mins
3	7	2025-01-04 11:00:00.000000 +00:00	2025-01-04 11:10:00.000000 +00:00	10 mins
4	8	2025-01-05 11:00:00.000000 +00:00	2025-01-05 11:10:00.000000 +00:00	10 mins

Figure 7. SQL query and result for Task 7.

Task 8 Airlines by country and creation date

SQL Query

```
SELECT *  
FROM airport_lab1.airline  
WHERE country IN ('France', 'Portugal', 'Poland')  
      AND created_at >= '2023-11-01'  
      AND created_at < '2024-04-01';
```

Result

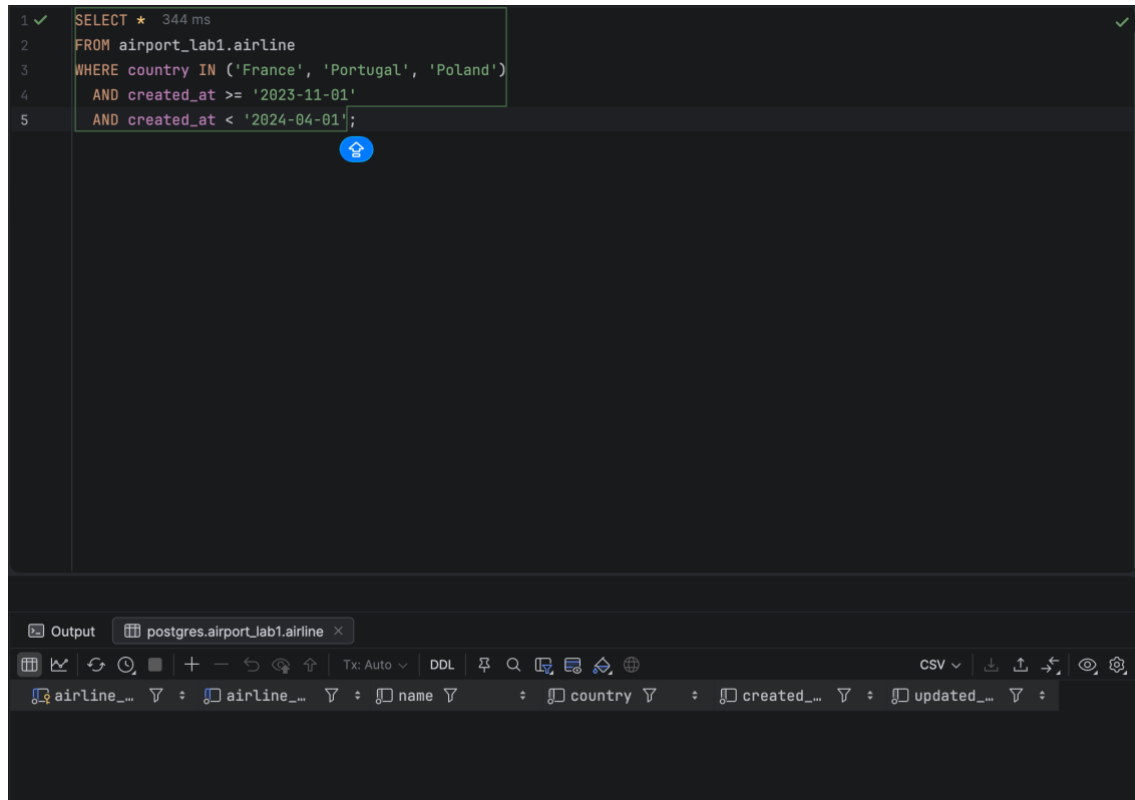


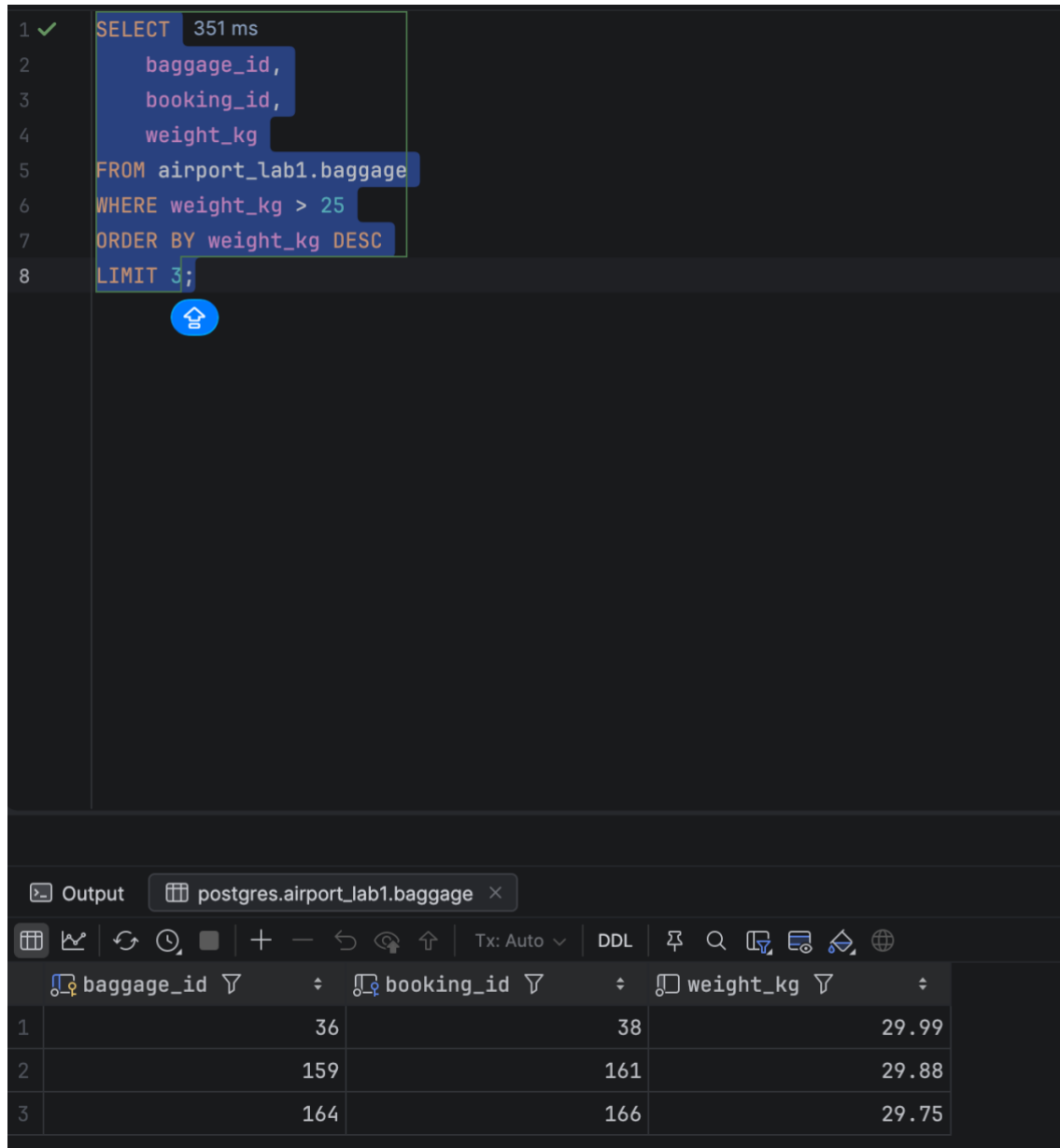
Figure 8. SQL query and result for Task 8.

Task 9 Three heaviest bags over 25 kg

SQL Query

```
SELECT
    baggage_id,
    booking_id,
    weight_kg
FROM airport_lab1.baggage
WHERE weight_kg > 25
ORDER BY weight_kg DESC
LIMIT 3;
```

Result



The screenshot shows a SQL IDE interface. The top pane displays the SQL query for selecting the three heaviest bags over 25 kg. The bottom pane shows the query results in a table format. The table has three columns: baggage_id, booking_id, and weight_kg. The results are ordered by weight_kg in descending order.

	baggage_id	booking_id	weight_kg
1	36	38	29.99
2	159	161	29.88
3	164	166	29.75

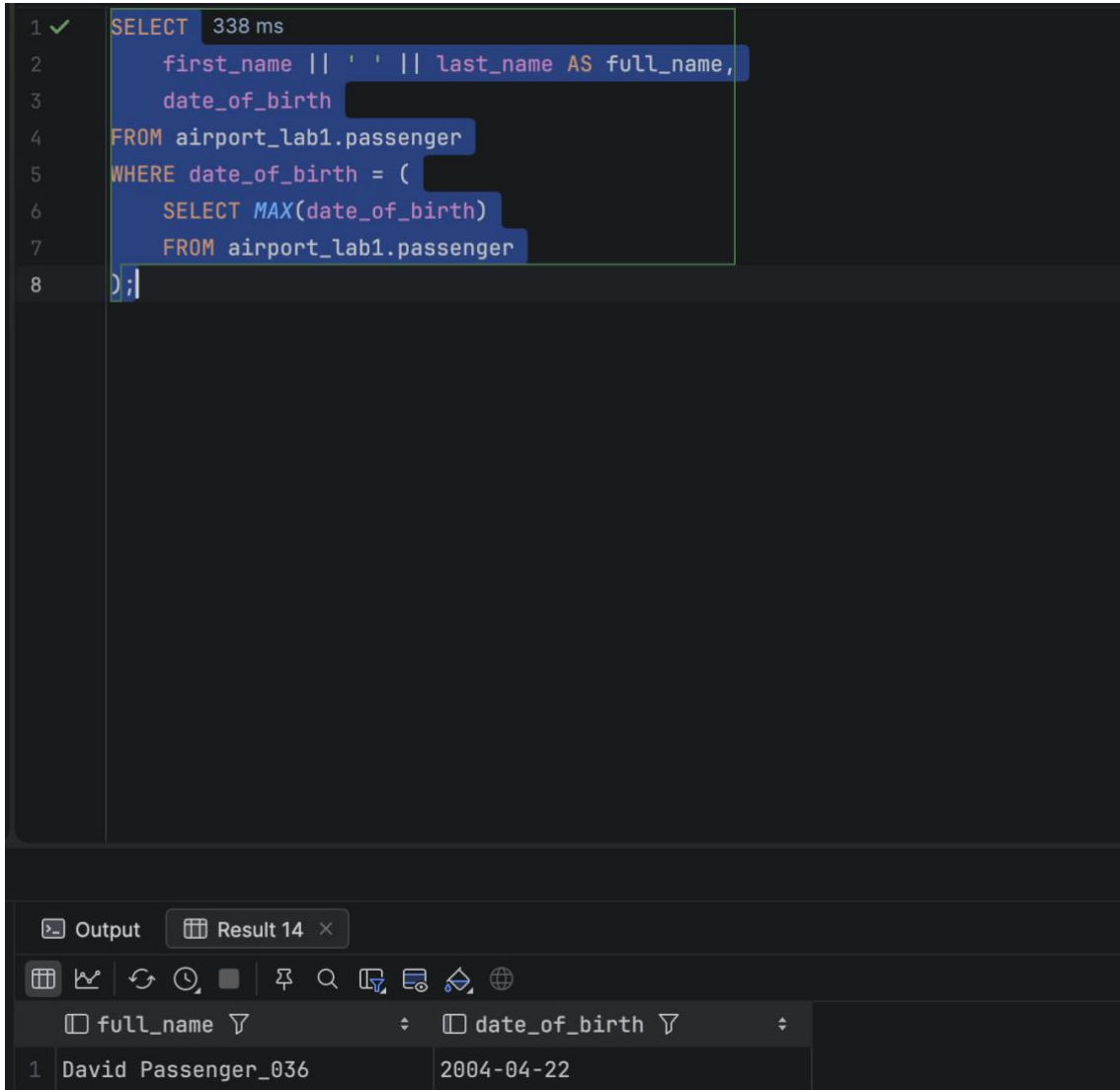
Figure 9. SQL query and result for Task 9.

Task 10 Youngest passengers

SQL Query

```
SELECT
    first_name || ' ' || last_name AS full_name,
    date_of_birth
FROM airport_lab1.passenger
WHERE date_of_birth = (
    SELECT MAX(date_of_birth)
    FROM airport_lab1.passenger
);
```

Result



The screenshot shows a SQL IDE interface. The top pane displays the following SQL query:

```
1 SELECT first_name || ' ' || last_name AS full_name,
2       date_of_birth
3 FROM airport_lab1.passenger
4 WHERE date_of_birth = (
5       SELECT MAX(date_of_birth)
6       FROM airport_lab1.passenger
7 );
8
```

The execution time is 338 ms. The bottom pane shows the result set with two columns: full_name and date_of_birth. The first row contains the values 'David Passenger_036' and '2004-04-22'.

	full_name	date_of_birth
1	David Passenger_036	2004-04-22

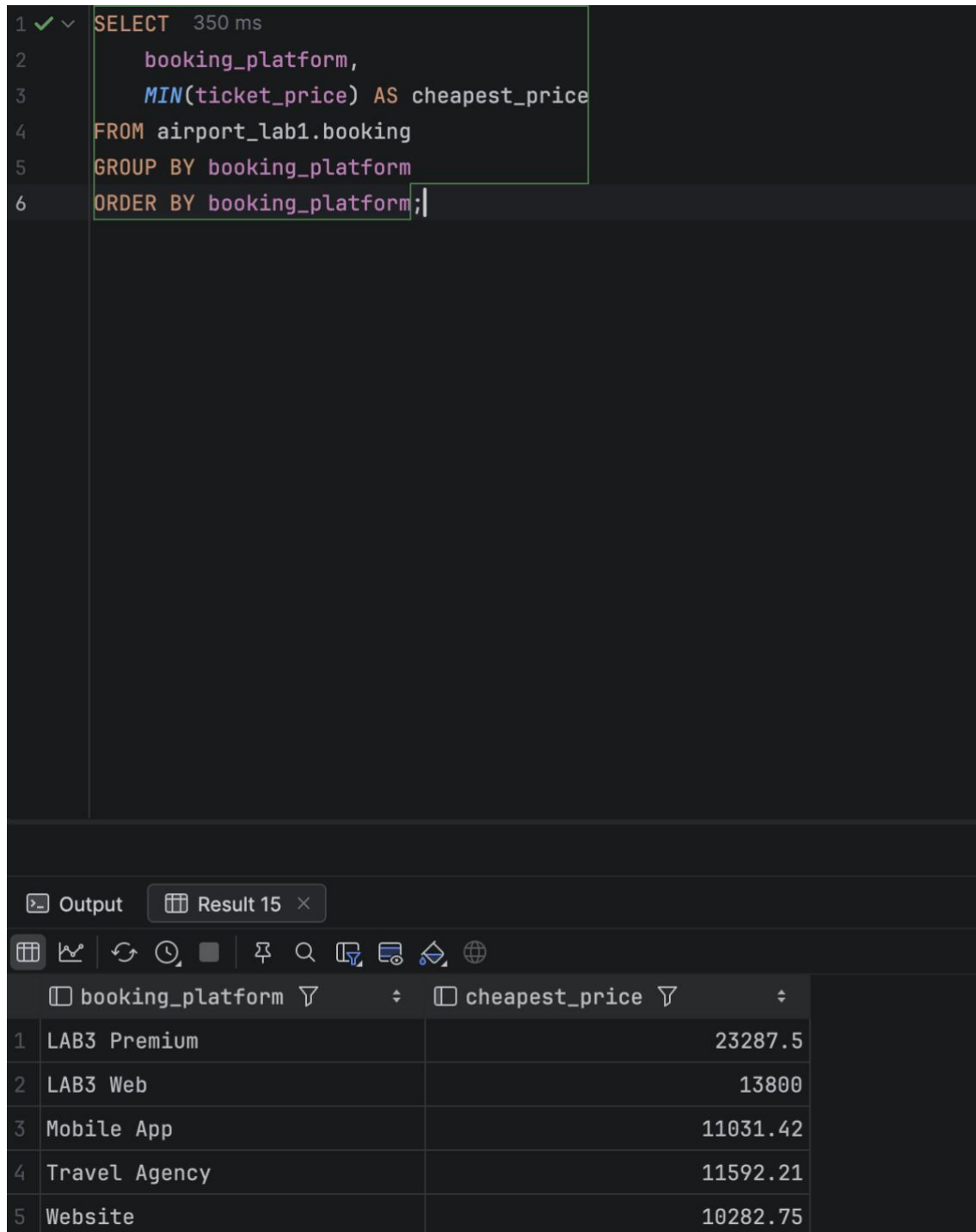
Figure 10. SQL query and result for Task 10.

Task 11 Lowest booking price by platform

SQL Query

```
SELECT
    booking_platform,
    MIN(ticket_price) AS cheapest_price
FROM airport_lab1.booking
GROUP BY booking_platform
ORDER BY booking_platform;
```

Result



The screenshot shows a SQL IDE interface. The top panel displays the SQL query: `SELECT booking_platform, MIN(ticket_price) AS cheapest_price FROM airport_lab1.booking GROUP BY booking_platform ORDER BY booking_platform;`. The bottom panel shows the results in a table with two columns: `booking_platform` and `cheapest_price`. The results are ordered by `booking_platform`.

	booking_platform	cheapest_price
1	LAB3 Premium	23287.5
2	LAB3 Web	13800
3	Mobile App	11031.42
4	Travel Agency	11592.21
5	Website	10282.75

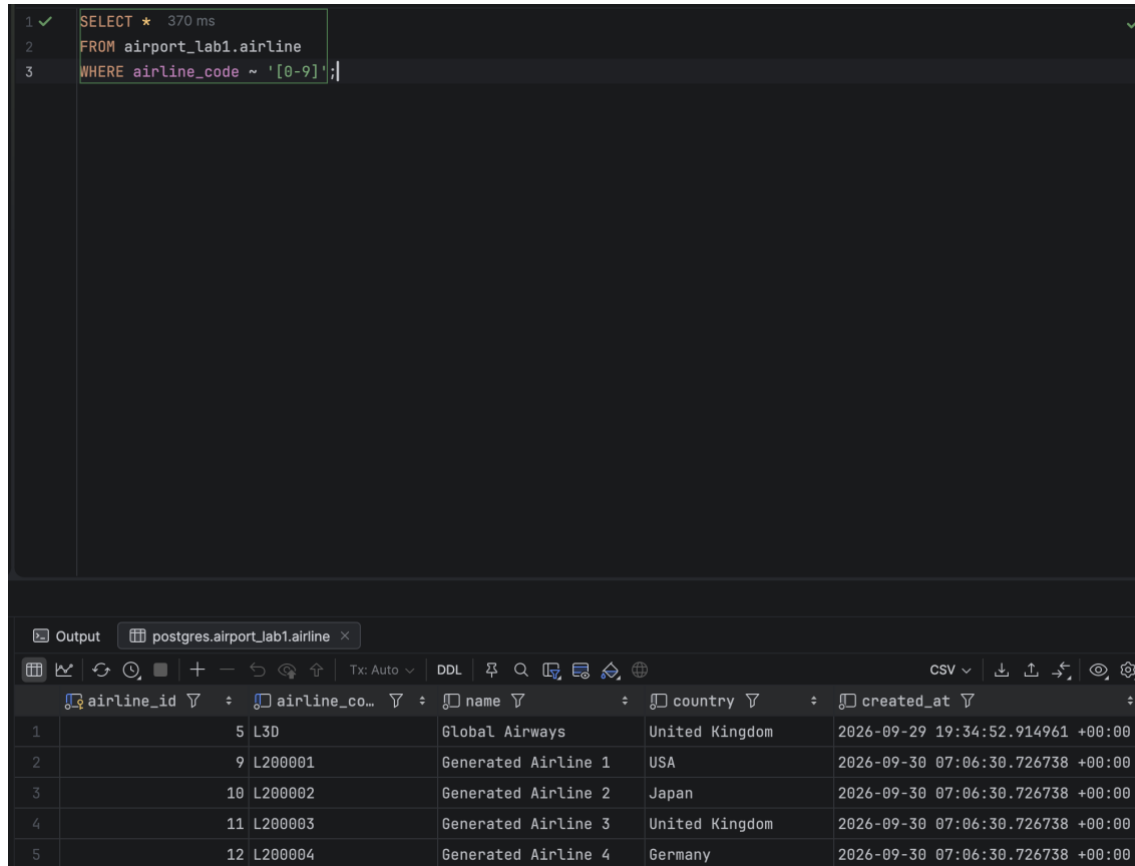
Figure 11. SQL query and result for Task 11.

Task 12 Airline codes containing a digit

SQL Query

```
SELECT *
FROM airport_lab1.airline
WHERE airline_code ~ '[0-9]';
```

Result



The screenshot shows a SQL query editor with the following query:

```
1 SELECT *
2 FROM airport_lab1.airline
3 WHERE airline_code ~ '[0-9]';
```

The query execution time is 370 ms. Below the query editor, the results are displayed in a table with the following columns: airline_id, airline_code, name, country, and created_at.

airline_id	airline_code	name	country	created_at
5	L3D	Global Airways	United Kingdom	2026-09-29 19:34:52.914961 +00:00
9	L200001	Generated Airline 1	USA	2026-09-30 07:06:30.726738 +00:00
10	L200002	Generated Airline 2	Japan	2026-09-30 07:06:30.726738 +00:00
11	L200003	Generated Airline 3	United Kingdom	2026-09-30 07:06:30.726738 +00:00
12	L200004	Generated Airline 4	Germany	2026-09-30 07:06:30.726738 +00:00

Figure 12. SQL query and result for Task 12.

Task 13 Five most recently created airlines

List the five most recently created airlines.

SQL Query and Result

```
SELECT *
FROM airport_lab1.airline
ORDER BY created_at DESC
LIMIT 5;
```

1 ✓
2
3
4

SELECT *
FROM airport_lab1.airline
ORDER BY created_at DESC
LIMIT 5;

Output postgres.airport_lab1.airline

Tx: Auto
DDL
Q
CSV

ne_id	airline_co...	name	country	created_at
209	3	yerow	Russia	2026-09-30 07:28:15.144822 +00:00
11	L200003	Generated Airline 3	United Kingdom	2026-09-30 07:06:30.726738 +00:00
10	L200002	Generated Airline 2	Japan	2026-09-30 07:06:30.726738 +00:00
9	L200001	Generated Airline 1	USA	2026-09-30 07:06:30.726738 +00:00
12	L200004	Generated Airline 4	Germany	2026-09-30 07:06:30.726738 +00:00

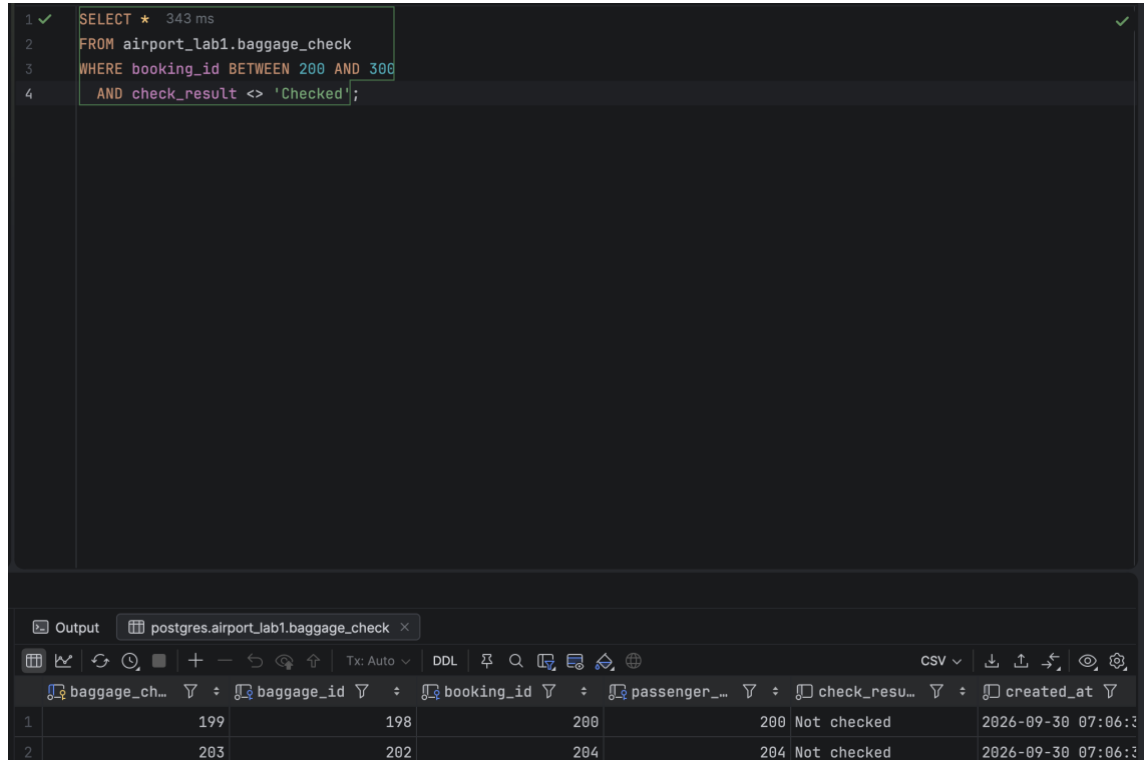
5 rows

Task 14 Baggage checks for bookings 200 to 300

SQL Query

```
SELECT *  
FROM airport_lab1.baggage_check  
WHERE booking_id BETWEEN 200 AND 300  
      AND check_result <> 'Checked';
```

Result



The screenshot shows a database query editor with a dark theme. The top panel displays the SQL query: `SELECT * FROM airport_lab1.baggage_check WHERE booking_id BETWEEN 200 AND 300 AND check_result <> 'Checked';`. The bottom panel shows the results of the query in a table format. The table has columns: `baggage_id`, `baggage_id`, `booking_id`, `passenger_name`, `check_result`, and `created_at`. The results are as follows:

	baggage_ch...	baggage_id	booking_id	passenger_...	check_resu...	created_at
1	199	198	200	200	Not checked	2026-09-30 07:06:3
2	203	202	204	204	Not checked	2026-09-30 07:06:3

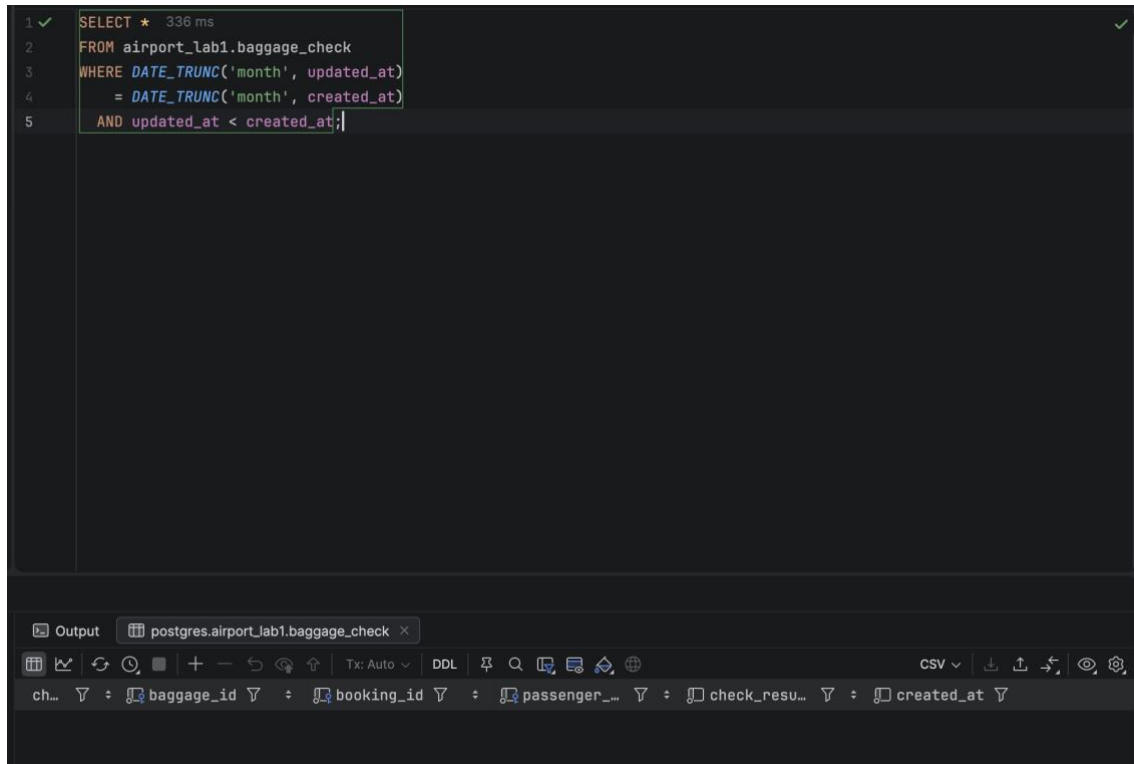
Figure 13. SQL query and result for Task 14.

Task 15 Baggage checks updated before creation

SQL Query

```
SELECT *  
FROM airport_lab1.baggage_check  
WHERE DATE_TRUNC('month', updated_at)  
      = DATE_TRUNC('month', created_at)  
      AND updated_at < created_at;
```

Result



The screenshot shows a SQL query execution interface. The query is displayed in a text editor with line numbers 1 to 5. The query is: `SELECT * FROM airport_lab1.baggage_check WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at) AND updated_at < created_at;`. The execution time is 336 ms. Below the query editor, there is a table with columns: `ch...`, `baggage_id`, `booking_id`, `passenger...`, `check_resu...`, and `created_at`. The table is currently empty.

Figure 14. SQL query and result for Task 15.